ITM Skills University · School of Future Tech

LeagueTrack

Local Sports League Score & Fixture Tracker App

B.Tech Computer Science Engineering and AI
Cross Platform Application · Viva preparation

Viva notes for the running Flutter app: problem statement, tech stack, Hive cache, and the path from a saved result to the standings table.

1. One-minute introduction

LeagueTrack is a Flutter app for a fictional local sports league. A fan opens Fixtures, filters upcoming, live, and completed matches, opens a match, and can record or update a full-time score. That save is not stored only on the screen. It goes through a repository, the standings calculator rebuilds the table from every finished fixture, Riverpod refreshes the providers, and both the match screen and the Standings screen update. If the mock fetch fails, Hive serves the last saved fixtures and standings.

2. Problem statement

LeagueTrack wants an app for a local sports league where fans view upcoming fixtures, check live and recent match scores, and see a standings table that updates as results are recorded.

The sheet also asks for a proper justification. The short justification is:

- Flutter is the cross-platform UI toolkit for this subject. One codebase runs on desktop, web, and mobile.
- The assignment asks us to simulate fetching with async/await. A mock API that waits with Future.delayed does that without a server.
- Riverpod is the state layer, so a saved result and the table stay in sync. The UI does not keep its own copy of the league.
- Hive caches fixtures and standings on the device, so the app can still open when the fetch fails. SharedPreferences stores only small settings such as the theme.
- The clubs are fictional local teams. That matches a local sports league and does not use real professional clubs.

3. Objectives and where they are in the app

Objective	Where it is satisfied
Fixtures, Match Detail, and Standings screens	FixturesScreen, MatchDetailsScreen, StandingsScreen. Bottom navigation switches Fixtures and Standings. A card opens match details.
ListView, DataTable-style layout, and Cards	Match details is a ListView of Cards. Standings is a DataTable inside a Card. Each fixture is a Card. The fixture list scrolls in a CustomScrollView.
Material 3 theme and team-color accents	AppTheme sets useMaterial3. Light and dark themes. Each club crest uses that club's primary and secondary colors.
async/await to simulate fetching	MockLeagueApi waits with Future.delayed, then returns the mock season. Fixtures shows a loading state until that future completes.
Riverpod keeps standings in sync	ResultRecorder saves through the repository, then invalidates the fixture and standings providers. The screens watch those providers.
Home, away, date, venue, and score if played	FixtureCard shows both clubs, the score or kickoff time, and a footer with the ground and the date.
Progress cues	Loading placeholders, Live / Upcoming / Full time badges, pull to refresh, and the Updated, Saved data, or Offline line.

4. Outcomes

- Fixtures shows FixtureLoadingView while the mock fetch is in progress.
- Recording a result calls StandingCalculator, and Riverpod rebuilds the table. Win = 3, draw = 1, loss = 0.
- Match details shows the scoreline, date, time, venue, and match statistics: goals, yellow cards, and red cards counted from that match's events.

• The first-ranked club is highlighted with a gold marker and a tinted row.

5. Deliverables

Deliverable	Status in this project
Fixture list, match detail, standings table	Done in lib/features.
Material 3 theme with team colors	Done in lib/core/theme.
Async fetch with Riverpod-synced standings	Done. MockLeagueApi plus providers.
Local storage for offline fixtures and standings	Done. Hive box local_league_cache.
Responsive fixtures-to-standings journey	Done. Phone and wide layouts, bottom navigation, and a table that scrolls sideways.
Figma design of the same flow	This is a design deliverable. It is not a file inside the Flutter project. The running app is the implemented flow.

6. What the user does, in order

• The app opens on Fixtures for the Local Sports League, season 2026/27.
• While the mock API is waiting, skeleton cards are shown.
• The list opens on the live matchweek when a match is in progress. Chips switch matchweek. A segmented control switches Upcoming, Live, and Completed.
• Every card shows the two clubs, the score if the match has started or finished, the ground, and the date.
• Tapping a card opens Match details: scoreline, date, time, venue, the record-result form, both clubs' current table rows, match statistics, and the event timeline.
• Save result marks the match full time, stores the score, recalculates the table, and the open match plus the fixture list show the new score.
• The Standings tab shows POS, TEAM, P, W, D, L, GF, GA, GD, PTS. Position 1 is highlighted. On a phone the remaining columns scroll sideways.
• A line under the title says when the data was last updated. A failed fetch says Saved data. An offline device with a snapshot says Offline.

7. Tech stack

Package name: league_track. App version 1.0.0+1. Dart SDK constraint ^3.13.2.

Piece	What it is used for
Flutter	UI framework. One project for desktop, web, and mobile.
Dart	Language for the UI, models, calculators, async API, and tests.
Material 3	Theme, NavigationBar, Cards, SegmentedButton, DataTable, buttons.
flutter_riverpod	State. Providers load data and refresh the UI after a result is saved.
MockLeagueApi	Simulated backend. Future.delayed stands in for a network call. There is no real server.
Hive and hive_flutter	Local database for one league snapshot: fixtures, standings, and clubs.
shared_preferences	Settings only: theme, favorite club, and followed clubs.
connectivity_plus	Tells the app whether the device currently has a network.
google_fonts 8.2.1	Plus Jakarta Sans for interface text and Barlow Condensed for scores.

Piece	What it is used for
intl	Fixed English date and time labels.
equatable	Value equality for the models.
flutter_test	Data tests and widget tests for loading, errors, offline, results, and the table.

8. Which database, and what is not a database

There is no MySQL, Firebase, SQLite server, or cloud database. The problem statement does not ask for a backend, login, or an admin website. Two local stores are used, and they do different jobs.

Hive is the league database

- Box name: local_league_cache. One string value, stored under the key snapshot.
- That string is JSON: fixtures, standings, clubs, and updatedAt. There are no generated Hive type adapters.
- It is written after a successful full fixture load, a successful standings load, and after a result is saved.
- If a later fetch throws, the repository reads this snapshot and the screens stay usable.
- An older box named league_cache is not opened, so a previous Premier League snapshot cannot appear.

SharedPreferences is only for settings

- It stores the theme choice, the favorite club id, and the list of followed clubs.
- It is not used for fixtures or the standings table.

The season itself starts in memory

MockLeagueData builds the season once when the process starts. FixtureFactory creates nine matchweeks from club strength with a fixed random seed, so the scores are realistic and repeatable. StandingCalculator then builds the table from the finished matches. The API keeps its own copies, so recording a result does not edit the original seed objects.

9. Architecture

The screens never call the mock API directly. The path is:

```
Screen
  -> Riverpod provider
    -> LeagueRepository
      -> MockLeagueApi     (simulated network)
      -> HiveLeagueCache    (local snapshot)
    -> StandingCalculator   (pure Dart, no widgets)
```

Folder	Responsibility
lib/main.dart	Opens SharedPreferences and Hive, then runs ProviderScope.
lib/app	MaterialApp, theme, and the bottom-navigation shell.
lib/features/fixtures	Fixtures screen, cards, loading, error, and offline views.
lib/features/matches	Match details, the score form, and ResultRecorder.
lib/features/standings	Standings screen and the DataTable.
lib/features/cache	The Updated / Saved data / Offline line.
lib/providers	Riverpod providers for the API, repository, cache, and settings.
lib/data/api	LeagueApi contract and MockLeagueApi.

Folder	Responsibility
lib/data/repositories	LeagueRepositoryImpl and PreferencesRepository.
lib/data/local	Hive cache, JSON snapshot, and the SharedPreferences store.
lib/data/mock	Eight clubs, squads, and generated fixtures.
lib/data/models	League, Team, Player, Fixture, Standing, MatchEvent, preferences.
lib/domain/standings	StandingCalculator. Points are calculated only here.
lib/core	Theme, spacing, colors, date formats, and connectivity.

10. How the app starts

- WidgetsFlutterBinding.ensureInitialized().
- SharedPreferences.getInstance() loads the settings store.
- Hive.initFlutter() and openBox('local_league_cache').
- ProviderScope overrides the in-memory defaults with those two real stores.
- LeagueTrackApp watches themeModeProvider and shows HomeShell.
- HomeShell keeps Fixtures and Standings alive together, so the table is already subscribed when a result is saved on a match screen.

11. Flow: open the app and read fixtures

- FixturesScreen watches fixtureBoardProvider.
- That provider reads isOnlineProvider.
- If the device is offline and Hive has fixtures, those fixtures are shown and the line says Offline.
- If the device is offline and Hive is empty, the screen says You are offline.
- If the device is online, the fixtures and teams providers call the repository.
- The repository calls MockLeagueApi. The API awaits Future.delayed (350 ms in the app, zero in tests) and then returns the data.
- A full, unfiltered fixture list is written into Hive. Standings and clubs are cached on their own successful loads.
- When the provider completes, the loading placeholders are replaced by cards.
- If the API throws and a snapshot exists, the repository returns the cached list and the line says Saved data.
- If the API throws and the cache is empty, the screen says Could not load fixtures and offers Try again.

12. Flow: record a result and update the table

This is the flow to walk through if the examiner asks how the table changes.

- The form holds only the numbers being typed. It does not own the league.
- Save calls ResultRecorder.record.
- If the device is offline, the recorder stops and the form says the result was not saved.
- If online, LeagueRepositoryImpl.recordResult calls MockLeagueApi.recordResult.
- The API rejects a negative score, finds the fixture, and replaces it with a finished copy: full time, both scores set, minute cleared.
- The API immediately runs StandingCalculator on every finished fixture and replaces its in-memory table.
- The repository then reads fixtures and standings again and writes both into Hive.

• If that follow-up read fails, the repository still updates the Hive snapshot: it replaces that one fixture and runs StandingCalculator on the cached fixtures, so the saved score is not lost.
• ResultRecorder invalidates the fixtures, standings, player-stats, and top-scorer providers.
• The fixture board, the standings board, and the open match watch those providers, so the scoreline, the fixture card, and the table all rebuild from the new data.

13. Standings rules

Only fixtures whose status is finished are counted. A live match stays out of the table until a full-time result is saved.

```
Points          = (wins x 3) + (draws x 1) + (losses x 0)
Played          = wins + draws + losses
Goal difference = goals for - goals against
```

Sort, highest first:

• Points, descending.
• If points are equal, goal difference, descending.
• If goal difference is also equal, goals scored, descending.
• If those are still equal, team id, so the order does not jump around.

A club on 20 points stays above a club on 19 even when the second club has a better goal difference. Two clubs on the same points are separated by goal difference, so +13 is above +11. The code is lib/domain/standings/standing_calculator.dart. The mock season and recordResult both call that class. The table is not a separate hardcoded list.

14. The mock season

League name: Local Sports League. Short name LSL. Country: India. Season: 2026/27. Eight clubs in a double round-robin would play 14 matchweeks. The app generates the first nine, including the start of the return fixtures. There are 4 matches in each matchweek, 36 fixtures, and 88 players.

Club	Ground
Mumbai Warriors	Shivaji Ground
Chembur United	Chembur Sports Complex
Govandi FC	Govandi Community Ground
Taloja Titans	Taloja Sports Arena
Navi Mumbai FC	Navi Mumbai Stadium
Kurla Rangers	Chembur Sports Complex
Bandra Athletic	Shivaji Ground
Thane United	Taloja Sports Arena

Some clubs share a ground because there are eight clubs and five local venues. Every player name is fictional. Each club has a squad of 11. Goals and cards are generated from club strength, so a scoreline matches the goal events that produced it. The mock clock is fixed at Monday 5 October 2026, 21:26, during the Monday match of matchweek 8. Matchweeks 1 to 7 are finished. Matchweek 8 has three finished matches and one live match kicked off at 20:00. Matchweek 9 is still scheduled.

15. Match details

• Scoreline when the match has started or finished. Kickoff time when it has not.
• Separate rows for date, time, and venue.
• Record result, or Update result once the match is already full time. Steppers change the score. Save goes through the repository.
• A table card shows each club's position, matches played, and points. Those numbers change when the result is saved.
• Match statistics count goals, yellow cards, and red cards from this fixture's events.
• The timeline lists minute, event type, player, and club. The timeline is the generated event list. The official score and the league table come from the saved result.

16. Offline line and last update

Situation	What the user sees
Fetch succeeded	Updated, with the date and time of the snapshot.
Online, but the mock API throws, and Hive has data	Saved data, with the snapshot time. Cards and the table stay on screen.
Offline, and Hive has data	Offline, with the snapshot time.
Offline or failed fetch, and Hive is empty	The large offline or error state, with Try again. No data is invented.

Pull to refresh checks connectivity again and reloads fixtures, clubs, and the screen provider. A refresh really calls the repository, and it can fall back to Hive.

17. Providers to name in the viva

Provider	Role
leagueApiProvider	The mock API. Tests can replace it or make every request fail.
leagueRepositoryProvider	API plus the Hive cache.
fixturesProvider	Fixture list. A full list is cached. A filtered request does not replace that full list.
standingsProvider	Current table from the repository.
fixtureBoardProvider	Fixtures plus clubs for the cards. Chooses cache when offline.
standingsBoardProvider	Table rows plus clubs for the DataTable.
matchDetailsProvider	One fixture, both clubs, the table, events, and players.
resultRecorderProvider	Saves a score, then invalidates the providers the screens watch.
leagueCacheProvider	Hive in the running app. An in-memory cache in tests.
cacheNoticeProvider	The text of the status line.
isOnlineProvider	True when connectivity_plus reports something other than no network.
themeModeProvider	System, light, or dark, read from SharedPreferences.

18. What was deliberately not built

Say this clearly if asked. These items are outside the problem statement.

• No admin panel and no separate admin website.
• No login, user accounts, or authentication.
• No Firebase, REST backend, chat, notifications, or payments.

• Record Result stays inside the Flutter app because the problem statement needs a recorded result to recalculate the table. It demonstrates that data flow. It is not a production login for a referee.

19. How to run it, and how it was checked

```
flutter run -d macos
flutter run -d chrome
flutter analyze
flutter test
```

In Android Studio the run configuration is main.dart. Choose macOS (desktop) or Chrome and press Run. flutter analyze reports no issues. The tests cover the generated season, the repository, saving a result, loading, error, and offline screens, the table sort, a Hive round-trip, a failed mock API, and a narrow phone layout.

20. Short answers

What is the app?

A local sports league tracker. Fans read fixtures and scores, open a match, and see a standings table that changes when a result is recorded.

Why Flutter?

The subject is a cross-platform application. Flutter draws the same Material 3 interface on desktop, web, and mobile from one Dart project.

Where is the backend?

There is no server. MockLeagueApi pretends to be one. Future.delayed is the network wait. That is the async/await fetch the objective asks for.

Which database?

Hive, on the device. One JSON snapshot of fixtures, standings, and clubs. SharedPreferences holds the theme and followed clubs. The season is generated in memory and copied into Hive after a successful load.

Why Hive rather than SharedPreferences for the table?

The snapshot is one larger document: every fixture and every standing row. Hive stores that document. SharedPreferences is for a few short settings.

How do you know the table is not hardcoded?

StandingCalculator walks finished fixtures only. The season is built with it, and recordResult runs it again after a score changes. A new win adds 3 points and 1 match played for the winner, and 0 points and 1 match played for the loser. A draw adds 1 point to each club. Tests check those changes.

What does Riverpod do here?

Screens watch providers. Providers watch the repository. After a save, the recorder invalidates the fixture and standings providers. Every watching screen rebuilds. The form does not update the table by itself.

What if the API fails?

The repository catches the error, reads Hive, and returns the last fixtures or standings. The status line says Saved data. If Hive is empty, the error screen is shown.

What if the phone is offline?

The screen providers read Hive and do not call the API. If a snapshot exists, it is shown with an Offline line. A new result is not saved while offline.

Why is a live match missing from the table?

The table is made from completed results. A live score can still change. Saving a full-time result marks that fixture finished, and the calculator includes it.

What is on a fixture card?

Home club, away club, the score if the match has been played, the date, and the venue. An upcoming match shows the kickoff time instead of a score.

How is first place highlighted?

The row whose calculated position is 1 gets a gold rank marker and a gold tint. The club in that row is whoever the sort puts first. It is not a fixed team.

Is this the Premier League?

No. The league is the Local Sports League: Mumbai Warriors, Chembur United, Govandi FC, Taloja Titans, Navi Mumbai FC, Kurla Rangers, Bandra Athletic, and Thane United.

During the viva, open Fixtures, open the live match, save a score, then open Standings and show that points, matches played, and rank changed.